

算法竞赛数论核心笔记：同余、逆元与欧拉函数

说明： 本文档整理自对话记录，涵盖了从“费马小定理”到“完全剩余系应用”，再到“逆元的三大求法”及“O(N) 组合数取模”的全部核心知识点与 C++ 实战代码。

1. 费马小定理与代数降阶

费马小定理是同余世界处理素数模数的基石，在竞赛中有两种主要形态：

- 形态一（条件成立）：若 p 为素数且 $\gcd(a, p) = 1$ ，则 $a^{p-1} \equiv 1 \pmod{p}$ 。
- 形态二（全局成立）：若 p 为素数，对任意整数 a ，均有 $a^p \equiv a \pmod{p}$ 。

CP 视角： 形态二极其重要，因为它无需判断互素（即无需 `if (a % p != 0)` 的特判），可以作为极其安全的“无脑降维前置化简器”。

2. 快速幂引擎的真实面貌

快速幂算法本身并没有利用费马小定理，它纯粹是利用了二进制倍增思想和同余的乘法性质： $(A \times B) \bmod m = ((A \bmod m) \times (B \bmod m)) \bmod m$ 。

```
// C++ 迭代版快速幂祖传模板
long long qpow(long long a, long long n, long long m) {
    a = (a % m + m) % m; // 祖传防负数兜底
    long long ans = 1 % m;
    while (n > 0) {
        if (n & 1) ans = (ans * a) % m; // 取最低位
        a = (a * a) % m; // 底数自我倍增
        n >>= 1; // 指数右移
    }
    return ans;
}
```

3. 欧拉函数：性质与工程落地

欧拉函数 $\varphi(x)$ 表示小于等于 x 且与 x 互素的正整数个数。核心计算公式（仅与质因数种类有关，与出现次数无关）：

$$\varphi(x) = x \cdot \prod (1 - 1/p_i)$$

3.1 $O(\sqrt{N})$ 单点求解

```
long long phi(long long x) {
    long long ans = x;
    for (long long i = 2; i * i <= x; ++i) {
        if (x % i == 0) {
            ans = ans / i * (i - 1); // 先除后乘防溢出
            while (x % i == 0) x /= i; // 榨干该质因数防重复
        }
    }
    if (x > 1) ans = ans / x * (x - 1); // 扫尾单身巨型质数
    return ans;
}
```

3.2 包含分支的底层逻辑： $\varphi(ax) = a \cdot \varphi(x)$

在欧拉筛打表时，当 $i \bmod p == 0$ ，等式 $\varphi(i \cdot p) = \varphi(i) \cdot p$ 成立。因为 p 的所有质因数已经存在于 i 的血脉中，新数字的质因数种类并未增加，故连乘公式的后半部分完全不变， p 作为常数被直接提取即可。

4. 模逆元：同余世界的除法三神兵

在同余世界中处理除法（例如组合数取模）必须转化为乘以逆元。求逆元有三大核心武器：

武器库	绝对前提条件	时间复杂度	战术评价
1. 费马小定理	模数 mod 必须是质数	$O(\log \text{mod})$	最强无脑首选。逆元为 $a^{p-2} \bmod p$ ，代码极短，90%题目适用。
2. 扩展欧几里得	a, m 互素	$O(\log m)$	通用型真神。无视模数性质，极其强悍，解决一般情况。
3. 欧拉定理	a, m 互素	$O(\sqrt{m} + \log m)$	特定场景奇招。逆元为 $a^{\varphi(m)-1} \bmod m$ 。

```
// 神仙级极简 exgcd
long long exgcd(long long a, long long b, long long &x, long long &y) {
    if (b == 0) { y = 0; x = 1; return a; }
    long long d = exgcd(b, a % b, y, x); // 巧妙倒转引用原地状态转移
    y -= a / b * x;
    return d;
}

// 祖传 exgcd 求逆元封装
long long mod_reverse_exgcd(long long a, long long m) {
    long long x, y;
    if (exgcd(a, m, x, y) != 1) return -1; // 不互素说明无逆元
    return (x % m + m) % m; // 绝杀护身符：处理返回的负数余数
}
```

5. 实战降维打击：O(N) 组合数取模模板

计算大规模的 $C_n^m \equiv n! \times (m!)^{-1} \times ((n-m)!)^{-1} \pmod{p}$ 时，通过等式 $1/i! = 1/(i+1)! \times (i+1)$ ，我们可以对最大阶乘仅求一次逆元，随后 $O(N)$ 线性倒推所有小阶乘的逆元。

```

const int MAXN = 1e5+5;
const int mod = 1e9+7;
long long mod_reverse[MAXN], factorial[MAXN];

void init(int upper) {
    factorial[0] = 1;
    for(int i = 1; i <= upper; ++i)
        factorial[i] = (factorial[i-1] * i) % mod;

    // 只对最大的阶乘求一次逆元  $O(\log p)$ 
    long long x, y;
    exgcd(factorial[upper], mod, x, y);
    mod_reverse[upper] = (x % mod + mod) % mod;

    //  $O(N)$  线性倒推所有小阶乘的逆元
    for(int i = upper - 1; i >= 0; --i) {
        mod_reverse[i] = (mod_reverse[i+1] * (i + 1)) % mod;
    }
}

long long caculate_comb(int n, int m) {
    if (m < 0 || m > n) return 0; // 考场保命防线
    return (factorial[n] * mod_reverse[m] % mod) * mod_reverse[n-m] % mod;
}

```

边界思考：为什么 0 也有逆元？

在纯数学同余世界里，数字 0 绝对没有逆元！但在这里， $\text{mod_reverse}[0] = 1$ 的真实含义是 0 的阶乘的逆元是 1。因为 $0! = 1$ ，所以 $(0!)^{-1} \equiv 1 \pmod{p}$ 。这是组合数学公式在 $m=0$ 时的防崩溃基石。